

UART Communication using AXI UART Lite and MicroBlaze Processor on Arty A7

Experiment Report:

Prepared By:

NAME: Chinmay N. Yalawatti

USN: 01FE24BEC258

Semester: 5th sem

Branch: E&C (E-div)

College: KLE Technological University BVB campus

Objective

To design and implement a UART communication system using the AXI UART Lite IP and MicroBlaze soft processor in Vivado IP Integrator and verify serial communication between the FPGA and a PC terminal.

Software and Hardware Requirements

Software

- Vivado Design Suite
- Vitis IDE
- Serial Terminal (Tera Term / PuTTY)

Hardware

- Arty A7 FPGA Board
- USB Cable
- PC/Laptop

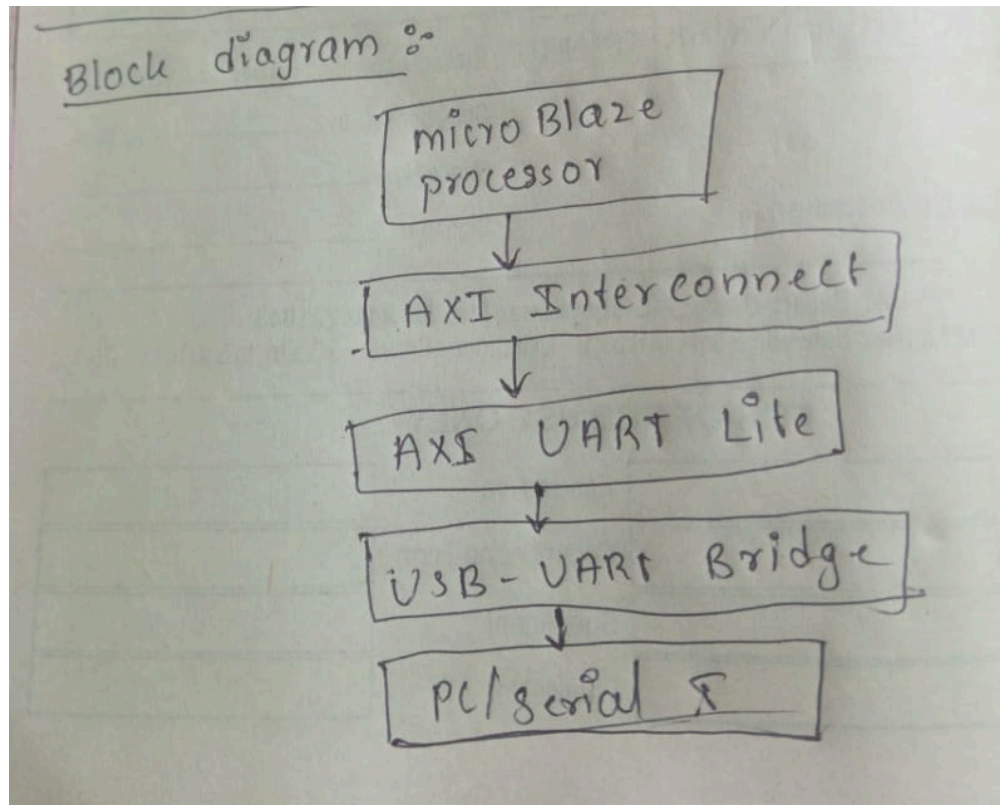
Theory

UART (Universal Asynchronous Receiver Transmitter) is a serial communication protocol widely used in embedded systems for communication between processors, sensors, computers, and peripheral devices.

The AXI UART Lite IP provides a simple UART interface that can be connected to the MicroBlaze processor through the AXI4-Lite bus.

In this experiment, a MicroBlaze processor transmits and receives data through the AXI UART Lite peripheral, which is connected to the USB-UART interface available on the Arty A7 board.

Block Diagram



Block-level explanation:

- **MicroBlaze Processor:** Soft-core RISC processor that runs the embedded application (transmit/receive/echo logic) and controls the AXI UART Lite through the AXI Interconnect.
- **AXI Interconnect:** Routes AXI4-Lite transactions between the MicroBlaze master and the AXI UART Lite (and other) slave peripherals.
- **AXI UART Lite:** Peripheral IP that performs the actual serial (UART) transmission/reception — parallel-to-serial and serial-to-parallel conversion — at the configured baud rate.
- **USB-UART Bridge:** On-board chip that converts the FPGA's UART TX/RX signals into a USB interface recognized by the PC as a virtual COM port.
- **PC / Serial Terminal:** Displays transmitted messages and sends typed characters back to the FPGA via a terminal program (Tera Term/PuTTY).

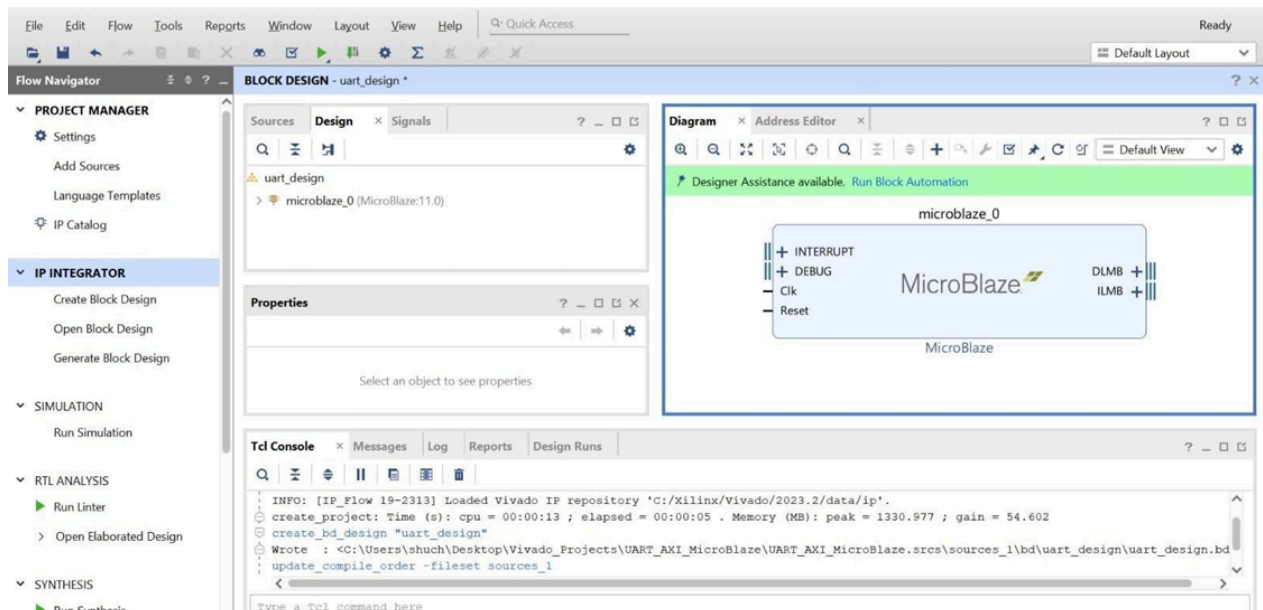
Problem Statement

Design a MicroBlaze-based embedded system using Vivado IP Integrator. Configure the AXI UART Lite peripheral for serial communication and develop an application that:

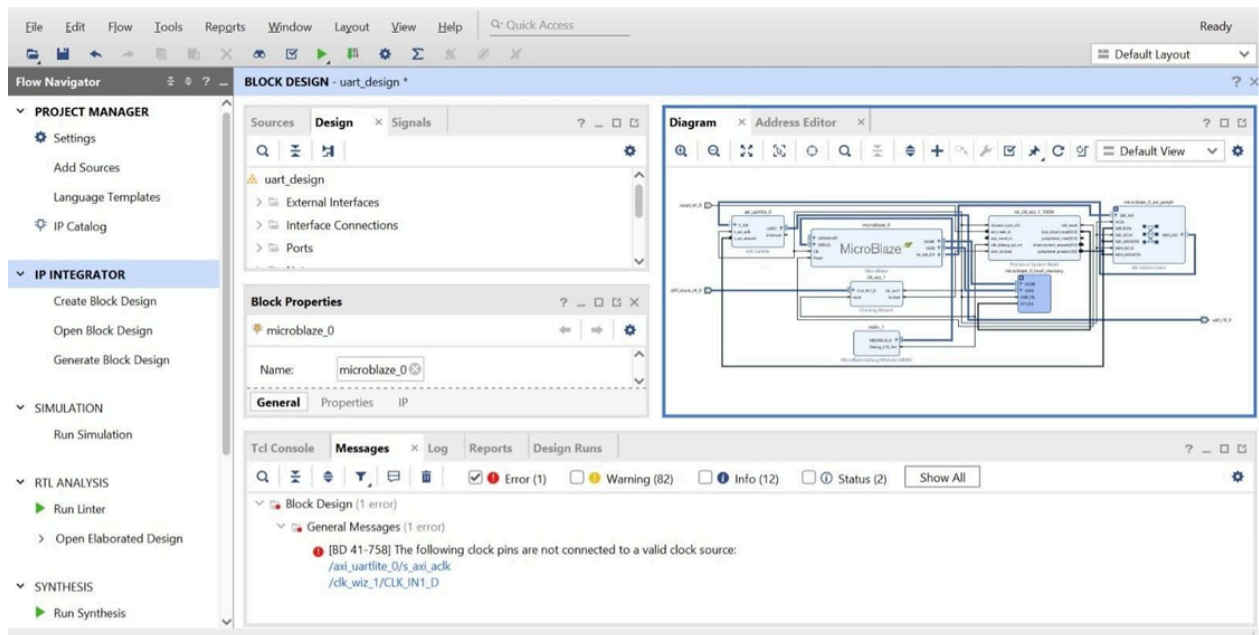
- 1 Transmits a predefined message.
- 2 Receives characters from the serial terminal.
- 3 Echoes the received characters back to the terminal.

Design Procedure

Part A: Hardware Design



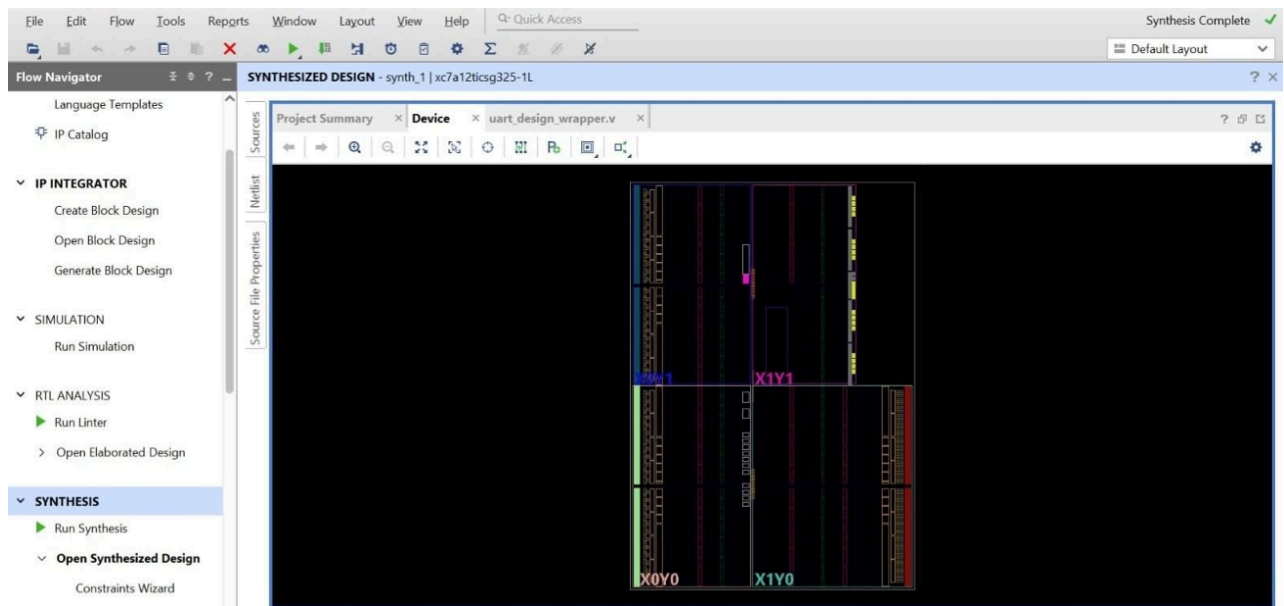
5. Connect all interfaces.



6. Validate the design.

7. Generate HDL Wrapper.

8. Generate Bitstream.



Synthesis completed successfully for the block design (device view, part xc7a12ticsg325-1L).

Part B: Software Development

Create a Vitis application project.

Write a program to:

- Send the message: Welcome to UART Communication
- Receive a character from the serial terminal.
- Echo the received character.

Sample Vitis application :

```
#include "xparameters.h"
#include "xil_printf.h"
#include <stdio.h>

int
main(void) {
    char rx_char;

    print("Welcome to UART Communication\r\n");

    while (1) { print("Enter a Character:\r\n");
        rx_char = inbyte(); /* receive one character */
        xil_printf("Received Character : %c\r\n",
            rx_char); }

    return 0;
}
```

Manual Calculation Questions — Solutions

Q9. Convert the following ASCII characters to hexadecimal representation.

Character	ASCII Hex
A	0x41
B	0x42
C	0x43
1	0x31

5	0x35
---	------

Q10. Convert the following hexadecimal values to ASCII characters.

Hex Value	Character
41	A
48	H
4C	L
4F	O

Q11. Calculate the total number of bits transmitted for the word: HELLO UART

frame per character = 1 Start Bit + 8 Data Bits + 1 Stop Bit = 10 bits/character

Word “HELLO” has 5 characters (H, E, L, L, O).

Total bits = 5 characters × 10 bits/character = 50 bits

UART Communication Verification

Test Case 1

Transmitted Message: Welcome to UART Communication

Expected Terminal Output: Welcome to UART Communication

Result: Pass — the terminal displays the message transmitted by the MicroBlaze application immediately after the board is powered on/reset, confirming correct UART transmit (TX) path operation.

Test Case 2

User Input: ABC123

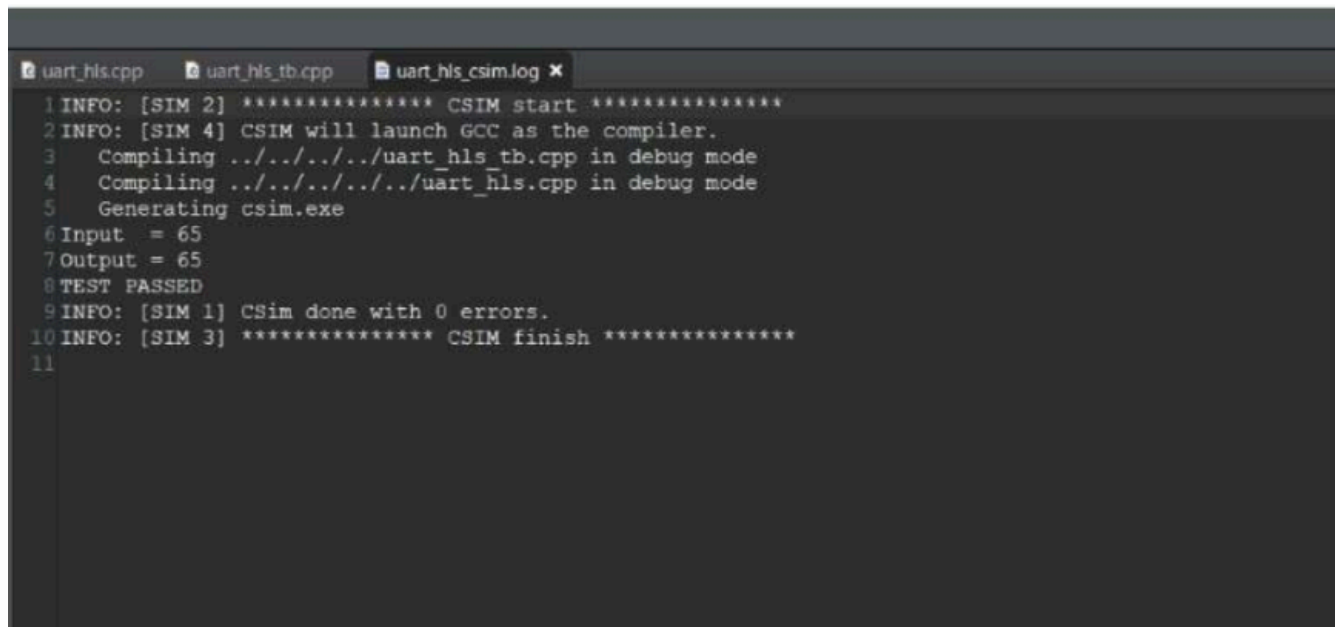
Expected Output: ABC123

Result: Pass — each character typed in the serial terminal is received by the AXI UART Lite (RX path), read by the MicroBlaze application, and echoed back character-by-character, confirming correct UART receive and echo operation.

Resource Analysis

Resource	Utilization
LUT	38
FF	46
BRAM	0
DSP	0

OUT PUT:



```
1 INFO: [SIM 2] ***** CSIM start *****
2 INFO: [SIM 4] CSIM will launch GCC as the compiler.
3   Compiling ../../../../uart_hls_tb.cpp in debug mode
4   Compiling ../../../../uart_hls.cpp in debug mode
5   Generating csim.exe
6 Input  = 65
7 Output = 65
8 TEST PASSED
9 INFO: [SIM 1] CSim done with 0 errors.
10 INFO: [SIM 3] ***** CSIM finish *****
11
```

Pre-Lab Questions — Solutions

Q1. What is UART?

UART (Universal Asynchronous Receiver Transmitter) is a hardware communication protocol/peripheral that performs serial, asynchronous (no shared clock signal) data transfer between two devices, converting parallel data from a processor into a serial bit stream for transmission and vice-versa for reception.

Q2. Expand UART.

UART — Universal Asynchronous Receiver Transmitter.

Q3. What are the basic UART signals?

- **TX (Transmit):** Line on which the device sends serial data out.
- **RX (Receive):** Line on which the device receives serial data.
- **GND (Ground):** Common reference/ground between the two communicating devices. (Optionally, hardware flow-control signals RTS/CTS are used in some UART implementations, though the basic AXI UART Lite only needs TX and RX.)

Q4. What is the purpose of baud rate?

Baud rate defines the rate (in symbols/bits per second) at which serial data is transmitted on the UART line. Since UART is asynchronous (no shared clock is transmitted), both the transmitter and receiver must be configured to the same baud rate so that the receiver samples each incoming bit at the correct time — a mismatch causes the received data to be misinterpreted.

Q5. What is the baud rate configured in this experiment?

The AXI UART Lite IP in this experiment is configured for the standard baud rate of 9600 bps (the default/commonly used setting for the IP; the actual value used should match what is set in the IP configuration and the serial terminal — record the value chosen in your Vivado IP customization).

Q6. What is the role of the MicroBlaze processor?

MicroBlaze is a soft-core RISC processor (implemented in FPGA fabric) that executes the embedded software application: it initializes the AXI UART Lite peripheral, transmits the predefined

message, polls/waits for received characters, and echoes them back — i.e., it provides the control/processing intelligence for the whole system.

Q7. What is the function of AXI UART Lite IP?

The AXI UART Lite IP implements the UART protocol in hardware — serializing outgoing data (parallel to-serial) for transmission and deserializing incoming data (serial-to-parallel) for reception — and exposes this functionality to the MicroBlaze processor through simple, memory-mapped AXI4-Lite registers (transmit register, receive register, status register, control register).

Q8. Which communication bus connects the processor and UART peripheral?

The AXI4-Lite bus (via the AXI Interconnect) connects the MicroBlaze processor to the AXI UART Lite peripheral.

Q12. Why is DSP utilization expected to be zero?

This design performs only serial data transfer, character storage, and simple control/comparison operations (no multiplication, filtering, or arithmetic-heavy signal processing). Since DSP slices are dedicated multiply-accumulate hardware and neither the MicroBlaze base configuration nor the AXI UART Lite IP requires hardware multipliers for this simple echo application, DSP utilization remains at 0.

Q13. Why are BRAM resources required in this design?

BRAM is required to implement the local memory for the MicroBlaze processor — it stores the compiled program instructions and data (stack, heap, variables) that MicroBlaze fetches and executes/accesses at runtime, via the AXI BRAM Controller.

Q14. Which IP block is responsible for UART communication?

The AXI UART Lite IP block is responsible for UART communication (serializing/deserializing data and interfacing with the USB-UART bridge).

Q15. Which IP block executes the software application?

The MicroBlaze (soft processor) IP block executes the software application.

Performance Analysis

Parameter	Value
System Clock Frequency	100 MHz
UART Baud Rate	9600 bps
Data Width	8 bits
Estimated Throughput	≈ 960 bytes/second

Q16. What happens if the terminal baud rate does not match the configured UART baud rate?

If the terminal's baud rate differs from the UART's configured baud rate, the receiver samples the incoming bits at the wrong time intervals, causing bit-level misalignment. This results in garbled/incorrect characters being displayed (or received), framing errors, or complete failure to detect

valid start/stop bits — reliable communication is not possible until both ends are set to the same baud rate.

Q17. Why is UART called asynchronous communication?

UART is called asynchronous because it does not use a separate, shared clock signal between the transmitter and receiver to synchronize data transfer. Instead, timing is derived independently at each end (based on a pre-agreed baud rate), and each data frame is bounded by its own start bit (signals the beginning of a frame) and stop bit(s) (signals the end), allowing the receiver to synchronize itself to each incoming frame without a common clock line.

Post-Lab Questions — Solutions

Q18. Compare UART and SPI communication.

Feature	UART	SPI
Communication Type	Asynchronous, serial	Synchronous, serial
Number of Wires	2 (TX, RX) + GND (point-to-point)	4 (MOSI, MISO, SCLK, SS/CS) minimum
Clock Signal	No shared clock line	Shared clock line (SCLK)
Speed	Lower (typically up to a few Mbps)	Higher (can reach tens of Mbps)

Q19. Mention three applications of UART communication in embedded systems.

- 1 Debug/console communication between a microcontroller/processor and a PC terminal (as in this experiment).
- 2 Communication with GPS modules, Bluetooth/Wi-Fi modules (e.g., HC-05, ESP8266) in embedded systems.
- 3 Communication between microcontrollers and sensors/peripheral modules (e.g., GSM modems, RFID readers) that provide a UART interface.

Q20. What are the advantages of using a soft processor such as MicroBlaze? •

Flexibility/customizability: The processor's configuration (cache sizes, peripherals, pipeline depth, number of instances) can be tailored exactly to the application's needs.

- **Integration:** It can be instantiated directly in the FPGA fabric alongside custom hardware accelerators, tightly coupled via AXI, without needing a separate physical processor chip.
- **Scalability:** Multiple MicroBlaze instances can be added if the application requires additional independent processing, limited only by available FPGA resources.
- **Portability across Xilinx FPGA families:** Since it's a soft IP core, the same design can be retargeted to different Xilinx FPGA devices/boards without needing a specific hard-processor SoC.